

Marketplace

Dossier technique du backend

État des lieux & guide de branchement du frontend

NestJS 11 · MongoDB (Mongoose 9) · Redis 7

Stripe · Cloudinary · Resend

Document généré le 3 août 2026

Branche `main` — 3 commits + travaux en cours

Comment lire ce document

Ce dossier a deux lecteurs en tête, et sert deux objectifs distincts :

- **Vous.** Comprendre précisément ce que votre backend fait aujourd'hui — les modèles, les règles métier, les transitions de statut — pour ne pas construire un frontend basé sur des suppositions.
- **Un agent IA** à qui vous confierez la construction du frontend. Les sections 5 (référence API), 7 (corrections) et 8 (structure recommandée) sont écrites pour être données telles quelles comme contexte.

État du backend

L'audit initial avait relevé huit points problématiques — dont trois bloquants pour le frontend (CORS absent, produits impossibles à publier, aucun « mes produits » vendeur). **Tous ont été corrigés et vérifiés par 39 tests de bout en bout** contre la pile réelle. La section 7 documente chaque correction et le raisonnement derrière. Le backend est prêt à recevoir un frontend.

Sommaire

1. Vue d'ensemble & pile technique
2. Architecture en modules
3. Modèles de données
 1. User · Category · Product
 2. Order & OrderItem
 3. Le panier : pourquoi il vit dans Redis
4. Authentification & autorisation
5. Référence complète de l'API
6. Les flux métier de bout en bout
7. Corrections appliquées
8. Structure de frontend recommandée

1. Vue d'ensemble & pile technique

Il s'agit d'une marketplace multi-vendeurs. Trois rôles cohabitent — `admin`, `seller`, `buyer` — et le catalogue distingue deux natures de produits : **physiques** (avec stock et expédition) et **numériques** (livrés par email après paiement). Cette distinction traverse tout le système : elle change le schéma du produit, le découpage des commandes, et le cycle de vie d'une commande.

Pile technique

Couche	Technologie	Rôle dans le projet
Framework	NestJS 11	Architecture modulaire, injection de dépendances, gardes
Base principale	MongoDB via Mongoose 9	Users, Categories, Products, Orders
Base éphémère	Redis 7 (ioredis)	Paniers, liste noire des tokens révoqués
Authentification	Passport JWT + bcryptjs	Token Bearer, expiration 7 jours
Validation	class-validator + class-transformer	<code>ValidationPipe</code> global et strict
Païement	Stripe (PaymentIntents)	Carte bancaire + webhook de confirmation
Images	Cloudinary	Upload en flux, stockage des URLs uniquement
Email	Resend	Bienvenue à l'inscription, livraison numérique
Conteneurisation	Docker Compose	redis + backend (:3000) + frontend (:5173)

MongoDB n'est pas dans le docker-compose

Seul Redis est conteneurisé. La base Mongo est attendue en externe via `MONGO_URI` — vraisemblablement un cluster MongoDB Atlas, au vu du format de l'exemple dans `.env.example`.

Configuration d'amorçage (`src/main.ts`)

```
const app = await NestFactory.create(AppModule, { rawBody: true });

app.useGlobalPipes(new ValidationPipe({
  whitelist: true,           // supprime tout champ non déclaré dans le DTO
  forbidNonWhitelisted: true, // ...et rejette carrément la requête (400)
  transform: true,          // convertit le JSON brut en instance de classe DTO
}));

await app.listen(3000);
```

Deux conséquences directes et importantes pour le frontend :

- `rawBody: true` conserve le `Buffer` brut de chaque requête. C'est indispensable à Stripe : `constructEvent()` vérifie la signature sur les octets exacts reçus, pas sur le JSON reparsé.
- `forbidNonWhitelisted: true` est **impitoyable**. Si votre formulaire React envoie ne serait-ce qu'un champ en trop — un `confirmPassword`, un `_id` renvoyé tel quel après un `GET` — l'API répond **400 Bad Request**. Nettoyez systématiquement vos payloads avant envoi.

CORS est activé

`app.enableCors({ origin: process.env.FRONTEND_URL || 'http://localhost:5173', credentials: true })`. Le `||` est délibéré : docker-compose définit une variable absente à la chaîne vide, que `??` ne rattraperait pas — l'origine autorisée serait alors `""`.

Variables d'environnement

Variable	Usage
<code>MONGO_URI</code>	Connexion Mongoose (chaîne Atlas)
<code>JWT_SECRET</code>	Signature des tokens — <code>getOrThrow</code> , l'app refuse de démarrer sans
<code>REDIS_HOST</code> / <code>REDIS_PORT</code>	Défauts <code>localhost</code> / <code>6379</code>
<code>CLOUDINARY_*</code>	<code>CLOUD_NAME</code> , <code>API_KEY</code> , <code>API_SECRET</code>
<code>STRIPE_SECRET_KEY</code>	Clé serveur Stripe
<code>STRIPE_WEBHOOK_SECRET</code>	Vérification de signature du webhook
<code>STRIPE_CURRENCY</code>	Devise, défaut <code>usd</code>
<code>RESEND_API_KEY</code> / <code>EMAIL_FROM</code>	Envoi des emails transactionnels

À noter : aucune variable `VITE_*` ni `FRONTEND_URL` n'existe encore. Il faudra en ajouter (au minimum l'URL publique de l'API et la clé publique Stripe) au moment de câbler le frontend.

2. Architecture en modules

Onze modules, dont deux marqués `@Global()` — `RedisModule` et `EmailModule`. Être global signifie que n'importe quel module peut injecter `RedisService` ou `EmailService` sans réimporter le module parent à chaque fois. C'est justifié ici : ces deux services sont transverses et utilisés par des fonctionnalités sans lien entre elles.

```
AppModule
├── ConfigModule (global)      variables d'environnement
├── MongooseModule.forRootAsync connexion Mongo différée jusqu'à ConfigService
├──
├── RedisModule               @Global    panier + liste noire des tokens
├── EmailModule               @Global    Resend : bienvenue, livraison numérique
├──
├── UsersModule               modèle User, aucun contrôleur exposé
├── AuthModule                ← Users    register / login / me / logout
├── CategoriesModule          arbre de catégories (admin)
├── ProductsModule            catalogue (seller)
├── UploadModule              Cloudinary
├── CartModule                ← Products, Redis
├── OrdersModule              ← Cart, Products, Users, Email
├── PaymentsModule            ← Orders    Stripe + espèces
```

Le sens des dépendances

La chaîne se lit de gauche à droite et ne remonte jamais : `Payments` → `Orders` → `Cart` → `Products`. Aucune dépendance circulaire, ce qui est sain — `ProductsService` ignore totalement l'existence des commandes.

Deux points de conception qui méritent d'être soulignés, parce qu'ils sont corrects et volontaires :

- **`UsersModule` n'expose aucun contrôleur.** Il n'existe donc aucune route `/users`. C'est un module de service pur, consommé par `Auth` et `Orders`. Les informations de l'utilisateur connecté passent exclusivement par `GET /auth/me`.
- **`markAsPaid()` vit dans `OrdersService`, pas dans `PaymentsService`.** C'est l'unique porte d'entrée par laquelle une commande devient payée, que le déclencheur soit le webhook Stripe ou la confirmation manuelle d'un paiement en espèces. Une seule règle métier, un seul endroit — les deux modes de paiement ne peuvent pas diverger.

Arborescence des sources

```
backend/src/
├─ main.ts, app.module.ts, app.controller.ts, app.service.ts
├─ common/
│   └─ decorators/    current-user.decorator.ts, roles.decorator.ts
│       └─ guards/    jwt-auth.guard.ts, roles.guard.ts
├─ auth/
│   └─ dto/           register.dto.ts, login.dto.ts
│       └─ strategies/ jwt.strategy.ts
├─ users/
│   └─ service, module + schemas/user.schema.ts
├─ categories/
│   └─ controller, service, module + dto/, schemas/
├─ products/
│   └─ controller, service, module + dto/, schemas/
├─ cart/
│   └─ controller, service, module + dto/
├─ orders/
│   └─ controller, service, module + dto/, schemas/
├─ payments/
│   └─ controller, service, module
├─ upload/
│   └─ controller, service, module
├─ email/
│   └─ service, module
└─ redis/
    └─ service, module, constants
```

Convention respectée partout : un dossier par domaine, contenant `*.controller.ts` / `*.service.ts` / `*.module.ts`, plus `dto/` pour les entrées validées et `schemas/` pour les modèles Mongoose. `cart/` n'a pas de `schemas/` — normal, son état est dans Redis.

3. Modèles de données

3.1 User

Collection `users`, horodatage automatique (`timestamps: true`).

Champ	Type	Contraintes & remarques
<code>email</code>	string	requis, unique, minuscules, trim
<code>passwordHash</code>	string	requis — hash bcrypt (coût 10). Le mot de passe brut n'est jamais stocké
<code>name</code>	string?	optionnel, trim
<code>role</code>	enum	<code>admin</code> <code>seller</code> <code>buyer</code> — défaut <code>buyer</code>

Le rôle est choisi librement à l'inscription

`RegisterDto` accepte `role: 'buyer' | 'seller'`. N'importe qui peut donc devenir vendeur en cochant une case. `admin` est exclu du DTO — un compte administrateur doit être créé manuellement en base. Pour un MVP c'est acceptable, mais gardez-le en tête.

3.2 Category

Arbre de catégories à profondeur libre, via un simple `parentId` auto-référencé.

Champ	Type	Contraintes & remarques
<code>name</code>	string	requis, trim
<code>slug</code>	string	requis, unique — généré automatiquement depuis <code>name</code>
<code>parentId</code>	ObjectId null	<code>null</code> = catégorie racine

Le `slugify()` maison passe en minuscules, retire les accents par normalisation `NFD` (é → e), et remplace tout caractère hors `a-z0-9` par un tiret. « Électroménager & Cuisine » devient `electromenager-cuisine`.

Le slug est généré mais jamais utilisé pour la lecture

Toutes les routes fonctionnent par `ObjectId` : `GET /categories/:id`. Il n'existe aucun `GET /categories/slug/:slug`. Si vous voulez des URLs propres côté frontend (`/c/electromenager`), il faudra soit ajouter cette route au backend, soit charger toutes les catégories au démarrage et résoudre le slug → id côté client. Vu leur faible nombre, la seconde option est tout à fait viable.

3.3 Product

C'est le schéma le plus riche, et le plus délicat : il fusionne deux natures de produits dans une seule collection. Les champs sont conditionnels selon `type`.

Champ	Type	Contraintes & remarques
<code>sellerId</code>	ObjectId → User	requis, indexé. Injecté depuis le JWT, jamais depuis le body
<code>categoryId</code>	ObjectId → Category	requis, indexé

Champ	Type	Contraintes & remarques
<code>title</code>	string	requis, trim
<code>description</code>	string?	optionnel
<code>price</code>	number	requis, ≥ 0
<code>promoPrice</code>	number?	optionnel, ≥ 0 — c'est lui qui est facturé quand il est défini et inférieur à <code>price</code>
<code>images</code>	string[]	URLs Cloudinary, défaut <code>[]</code> — jamais de binaire en base
<code>status</code>	enum	<code>draft</code> <code>active</code> <code>suspended</code> — défaut <code>draft</code>
<code>type</code>	enum	requis — <code>physical</code> <code>digital</code>
Réservé aux produits physiques		
<code>stock</code>	number?	≥ 0 — décrémenté atomiquement au checkout, restitué à l'annulation
<code>weightGrams</code>	number?	≥ 0
<code>shippingOptions</code>	{label, cost}[]?	Renseignable via le DTO (validation imbriquée). Facultatif même en physique
Réservé aux produits numériques		
<code>fileKey</code>	string?	Référence de stockage — ne doit jamais être une URL publique
<code>downloadLimit</code>	number?	≥ 1 — stocké, mais aucun compteur ne l'applique

La validation conditionnelle

`CreateProductDto` utilise `@ValidateIf()` pour n'exiger les champs que lorsqu'ils sont pertinents :

```
@ValidateIf((dto) => dto.type === 'physical')
@IsInt() @Min(0)
stock?: number;

@ValidateIf((dto) => dto.type === 'digital')
@IsString() @IsNotEmpty()
fileKey?: string;
```

Le frontend doit refléter exactement cette logique : un formulaire produit où le choix physique / numérique bascule deux jeux de champs distincts. Envoyer `stock` sur un produit numérique déclenchera une erreur 400, puisque `forbidNonWhitelisted` est actif.

3.4 Order & OrderItem

Une commande représente **un vendeur et une seule nature de produit**. C'est la décision structurante de tout le système de commandes : un panier contenant des articles de trois vendeurs différents produit trois commandes distinctes.

OrderItem — un instantané figé

Sous-document déclaré avec `@Schema({ _id: false })` : les lignes ne sont jamais éditées ni recherchées individuellement, elles n'ont donc pas besoin d'identité propre.

Champ	Type	Remarques
<code>productId</code>	ObjectId → Product	requis
<code>title</code>	string	Copie du titre <i>au moment de l'achat</i>
<code>price</code>	number	Copie du prix <i>au moment de l'achat</i>
<code>quantity</code>	number	≥ 1

Pourquoi dupliquer titre et prix plutôt que faire un populate ?

Parce qu'une commande est un document historique. Si le vendeur baisse son prix la semaine suivante, une commande passée ne doit pas voir son montant réécrit rétroactivement. Le `total` stocké resterait alors incohérent avec la somme de ses lignes. C'est un bon réflexe de conception, et il est appliqué correctement ici.

Order

Champ	Type	Remarques
<code>buyerId</code>	ObjectId → User	requis, indexé
<code>sellerId</code>	ObjectId → User	requis, indexé
<code>fulfillmentType</code>	enum	<code>physical</code> <code>digital</code> — une seule valeur, jamais mixte
<code>items</code>	OrderItem[]	requis
<code>total</code>	number	Σ (price × quantity), calculé au checkout
<code>paymentMethod</code>	enum	<code>stripe</code> <code>cash</code>
<code>stripePaymentIntentId</code>	string?	Renseigné à la création de l'intent
<code>status</code>	enum	6 valeurs — défaut <code>pending_payment</code>

3.5 Le panier : pourquoi il vit dans Redis

Le panier n'a **aucune collection MongoDB**. Il est stocké sous la clé `cart:<userId>` avec une durée de vie de **7 jours**, réinitialisée à chaque écriture. La valeur stockée est volontairement minimale :

```
// clé : cart:66f1a2b3c4d5e6f7a8b9c0d1
{ "66a1...": 2, "66b2...": 1 } // productId → quantité
```

À la lecture, `getCart()` enrichit cette carte brute en allant chercher `title`, `price` et `images` en base pour chaque produit, puis calcule les sous-totaux. Le panier renvoie donc toujours des **prix à jour**, jamais un instantané périmé — exactement l'inverse de `OrderItem`, et c'est volontaire dans les deux cas.

Autre détail soigné : si un produit a été supprimé entre-temps, `findOne()` lève une `NotFoundException`, qui est **rattrapée**. L'article est retiré du panier, le panier est resauvegardé, et la requête aboutit normalement au lieu de planter.

```
try {
  const product = await this.productsService.findOne(productId);
  items.push({ /* ... */ });
} catch {
  delete cart[productId]; // produit disparu → on l'oublie silencieusement
  cartChanged = true;
}
```

Conséquence directe pour le frontend

Le panier est **lié au compte utilisateur**, pas au navigateur. Un visiteur non connecté ne peut rien mettre au panier : toutes les routes `/cart` exigent un JWT. Deux options s'offrent à vous — soit exiger la connexion avant tout ajout, soit maintenir un panier local (`localStorage`) fusionné avec le panier serveur au moment du login. La seconde offre une bien meilleure expérience, mais c'est du travail purement frontend : le backend ne vous aidera pas.

4. Authentification & autorisation

Le parcours d'un token

1. `POST /auth/register` ou `/auth/login` renvoie `{ access_token }`.
2. La charge utile signée est minimale : `{ sub: userId, role }`, valable **7 jours**.
3. Le frontend le renvoie ensuite sur chaque appel protégé : `Authorization: Bearer <token>`.
4. `JwtStrategy.validate()` vérifie la signature, l'expiration, **puis la liste noire Redis**, et pose `{ userId, role }` sur `req.user`.
5. Les contrôleurs y accèdent via le décorateur `@CurrentUser()`.

Aucun refresh token

Un seul token, 7 jours, non renouvelable. Passé ce délai, l'utilisateur est déconnecté sans préavis. Le frontend doit intercepter les **401** globalement, purger l'état d'authentification et rediriger vers la page de connexion.

Les deux gardes

Garde	Comportement
<code>JwtAuthGuard</code>	Étend <code>AuthGuard('jwt')</code> de Passport. Rejette en 401 si le token est absent, mal signé, expiré ou révoqué.
<code>RolesGuard</code>	Lit les métadonnées posées par <code>@Roles(...)</code> via le <code>Reflector</code> . Sans <code>@Roles()</code> sur la route, il laisse passer tout utilisateur authentifié. Rejette en 403 sinon.

L'ordre de déclaration compte

`@UseGuards(JwtAuthGuard, RolesGuard)` — dans cet ordre précis. `RolesGuard` lit `request.user.role`, qui n'existe que si `JwtAuthGuard` s'est exécuté avant. Inversés, `user` serait `undefined` et l'accès serait refusé systématiquement.

La révocation à la déconnexion

Un JWT est par nature sans état : le serveur ne peut pas « l'oublier ». Une déconnexion qui se contenterait de supprimer le token côté navigateur laisserait un token parfaitement valide pendant 7 jours à quiconque en aurait fait une copie. La parade retenue est une liste noire dans Redis :

```
async logout(token: string) {
  const decoded = this.jwtService.decode(token) as { exp?: number } | null;
  const ttlSeconds = decoded?.exp ? decoded.exp - Math.floor(Date.now() / 1000) : 0;

  if (ttlSeconds > 0) {
    await this.redisService.set(`auth:blacklist:${token}`, '1', ttlSeconds);
  }
  return { message: 'Déconnecté avec succès' };
}
```

La finesse est dans le TTL : la clé Redis expire **exactement** quand le token lui-même expire. La liste noire ne peut donc pas grossir indéfiniment — elle se purge seule. Côté vérification, `JwtStrategy` consulte cette liste à chaque requête :

```

async validate(req: Request, payload: JwtPayload) {
  const token = ExtractJwt.fromAuthHeaderAsBearerToken()(req);
  const blacklisted = token && (await this.redisService.get(`auth:blacklist:${token}`));
  if (blacklisted) {
    throw new UnauthorizedException('Session expirée, veuillez vous reconnecter');
  }
  return { userId: payload.sub, role: payload.role };
}

```

C'est `passReqToCallback: true` dans le constructeur de la stratégie qui donne accès à la requête brute, et donc au token d'origine — la charge décodée seule ne suffirait pas.

Les emails transactionnels

Deux messages sont envoyés via Resend, tous deux en français :

Méthode	Déclencheur	Contenu
<code>sendWelcomeEmail()</code>	<code>AuthService.register()</code>	Message de bienvenue, personnalisé avec le prénom si fourni
<code>sendDigitalDelivery()</code>	<code>OrdersService.markAsPaid()</code> , uniquement si <code>fulfillmentType === 'digital'</code>	Un email par article , contenant le lien de téléchargement

L'envoi d'email n'est pas protégé contre l'échec

`register()` fait `await this.emailService.sendWelcomeEmail(...)` sans `try/catch`. Si Resend est indisponible ou la clé API invalide, **l'inscription entière échoue** alors que le compte vient d'être créé en base. L'utilisateur voit une erreur mais son compte existe — il ne pourra pas se réinscrire (409 Conflict). Un `try/catch` avec journalisation serait préférable : l'email de bienvenue n'est pas critique au point de bloquer une inscription.

5. Référence complète de l'API

Base : `http://localhost:3000` — **aucun préfixe global**. Les routes sont exposées à la racine (`/auth/login` , et non `/api/auth/login`). Légende : **PUBLIC** aucune authentification · **JWT** token requis · **RÔLE** rôle spécifique exigé.

5.1 Authentification — `/auth`

Méthode	Route	Accès	Corps & réponse
POST	<code>/auth/register</code>	PUBLIC	<code>{ email, password (≥8), name?, role? }</code> → <code>{ access_token }</code> . 409 si l'email existe déjà. Envoie l'email de bienvenue.
POST	<code>/auth/login</code>	PUBLIC	<code>{ email, password }</code> → <code>{ access_token }</code> . 401 si identifiants invalides.
GET	<code>/auth/me</code>	JWT	→ <code>{ id, email, name, role }</code> . Jamais le <code>passwordHash</code> .
POST	<code>/auth/logout</code>	JWT	Aucun corps — le token est lu dans l'en-tête <code>Authorization</code> . → <code>{ message }</code> . Révoque le token jusqu'à son expiration.

5.2 Catégories — `/categories`

Méthode	Route	Accès	Corps & réponse
POST	<code>/categories</code>	JWT admin	<code>{ name, parentId? }</code> — le <code>slug</code> est généré côté serveur
GET	<code>/categories</code>	PUBLIC	Liste à plat — l'arbre est à reconstruire côté client via <code>parentId</code>
GET	<code>/categories/:id</code>	PUBLIC	Par ObjectId uniquement, pas par slug
PATCH	<code>/categories/:id</code>	JWT admin	Renommer régénère le slug. Refuse qu'une catégorie soit son propre parent
DELETE	<code>/categories/:id</code>	JWT admin	400 si la catégorie contient des sous-catégories ou des produits. Le message précise le nombre concerné

5.3 Produits — `/products`

Méthode	Route	Accès	Corps & réponse
POST	<code>/products</code>	JWT seller	<code>CreateProductDto</code> . Le <code>sellerId</code> vient du token. Créé en <code>status: 'draft'</code>
GET	<code>/products</code>	PUBLIC	Catalogue publié uniquement (<code>status: 'active'</code>). Paramètres : <code>categoryId</code> , <code>sellerId</code> , <code>q</code> , <code>page</code> (déf. 1), <code>limit</code> (déf. 20, max 100). → <code>{ items, total, page, limit, totalPages }</code> . <code>categoryId</code> et <code>sellerId</code> sont peuplés
GET	<code>/products/mine</code>	JWT seller	Mes produits, tous statuts confondus — brouillons inclus. Tableau simple, non paginé
GET	<code>/products/:id</code>	PUBLIC	Fiche produit enrichie (catégorie + vendeur peuplés). Renvoie le produit quel que soit son statut
PATCH	<code>/products/:id</code>	JWT seller	Filtré sur <code>{ _id, sellerId }</code> : un vendeur ne peut modifier que ses produits

Méthode	Route	Accès	Corps & réponse
PATCH	/products/:id/status	JWT seller	{ status: 'draft' 'active' } — publier / dépublier. 'suspended' est refusé (400) : c'est un statut de modération
DELETE	/products/:id	JWT seller	Même filtre de propriété. Réponse vide

Sur `PATCH` et `DELETE`, un produit appartenant à un autre vendeur renvoie **404** et non 403 — le filtre porte sur `sellerId`, donc rien n'est trouvé. C'est défendable (on ne révèle pas l'existence de la ressource), mais le message d'erreur, lui, précise « ou vous n'en êtes pas le vendeur », ce qui annule ce bénéfice.

5.4 Upload — /upload

Méthode	Route	Accès	Corps & réponse
POST	/upload/image	JWT seller	multipart/form-data, champ nommé <code>image</code> → { url } . Dossier Cloudinary <code>marketplace/products</code>

Route protégée

Réservée aux vendeurs authentifiés, plafonnée à **5 Mo**, et limitée aux types `image/jpeg`, `image/png` et `image/webp`. Un format refusé renvoie un **400** explicite plutôt qu'un silencieux « aucun fichier reçu ».

5.5 Panier — /cart

`@UseGuards(JwtAuthGuard)` est posé sur la classe entière : toutes les routes exigent un token.

Méthode	Route	Corps & réponse
GET	/cart	→ <code>{ items: [{ productId, quantity, title, price, originalPrice?, images, subtotal }], total }</code> . <code>price</code> est le montant réellement facturé ; <code>originalPrice</code> n'apparaît qu'en cas de promotion, pour le prix barré
POST	/cart/items	<code>{ productId, quantity ≥ 1 }</code> . Quantité ajoutée à l'existante. 404 si le produit n'existe pas. Renvoie le panier complet
PATCH	/cart/items/:productId	<code>{ quantity ≥ 0 }</code> . Quantité remplacée . 0 retire l'article. Renvoie le panier complet
DELETE	/cart/items/:productId	Retire l'article. Renvoie le panier complet
DELETE	/cart	Vide le panier → <code>{ items: [], total: 0 }</code>

Bonne nouvelle pour la gestion d'état

Les cinq routes renvoient le panier complet recalculé. Vous n'avez donc jamais besoin de refaire un `GET /cart` après une mutation : la réponse est l'état à jour. Avec RTK Query, cela se traduit par une mise à jour directe du cache (`updateQueryData`) plutôt qu'une invalidation suivie d'un rechargement.

5.6 Commandes — /orders

JWT exigé sur toute la ressource.

Méthode	Route	Rôle	Comportement
POST	/orders/checkout	—	<code>{ paymentMethod: 'stripe' 'cash' }</code> → un tableau de commandes. 400 si le panier est vide ou si le stock est insuffisant (message « Stock insuffisant pour « ... » »). Réserve le stock puis vide le panier
GET	/orders/my-purchases	—	Mes achats, triés du plus récent au plus ancien
GET	/orders/my-sales	seller	Mes ventes, même tri
GET	/orders/:id	—	Accessible à l'acheteur <i>ou</i> au vendeur. 403 pour un tiers
PATCH	/orders/:id/ship	seller	<code>processing</code> → <code>shipped</code> . Produits physiques uniquement
PATCH	/orders/:id/confirm-delivery	—	Acheteur seul : <code>shipped</code> → <code>delivered</code>
PATCH	/orders/:id/cancel	—	Acheteur ou vendeur. Uniquement depuis <code>pending_payment</code> ou <code>processing</code>

5.7 Paiements — /payments

Méthode	Route	Accès	Comportement
POST	/payments/:orderId/create-intent	JWT	Acheteur uniquement. → <code>{ clientSecret }</code> à passer à Stripe.js. Exige <code>paymentMethod: 'stripe'</code> et <code>status: 'pending_payment'</code>

Méthode	Route	Accès	Comportement
PATCH	/payments/:orderId/confirm-cash	JWT seller	Le vendeur atteste avoir reçu les espèces → déclenche <code>markAsPaid()</code>
POST	/payments/webhook	PUBLIC	Appelé par Stripe. Jamais par le frontend. La vérification de signature est la sécurité de cette route

6. Les flux métier de bout en bout

6.1 Le découpage du panier en commandes

C'est la mécanique la plus importante à comprendre avant de dessiner l'écran de commande. `checkout()` regroupe les articles du panier par clé composite `'${sellerId}:${type}'`, et crée **une commande par groupe**.

Panier de l'acheteur

- |— Clavier mécanique — vendeur A, physique
- |— Souris gamer — vendeur A, physique
- |— Ebook « NestJS » — vendeur A, numérique
- |— Tapis de souris — vendeur B, physique

↓ POST /orders/checkout

3 commandes distinctes, chacune avec son propre statut :

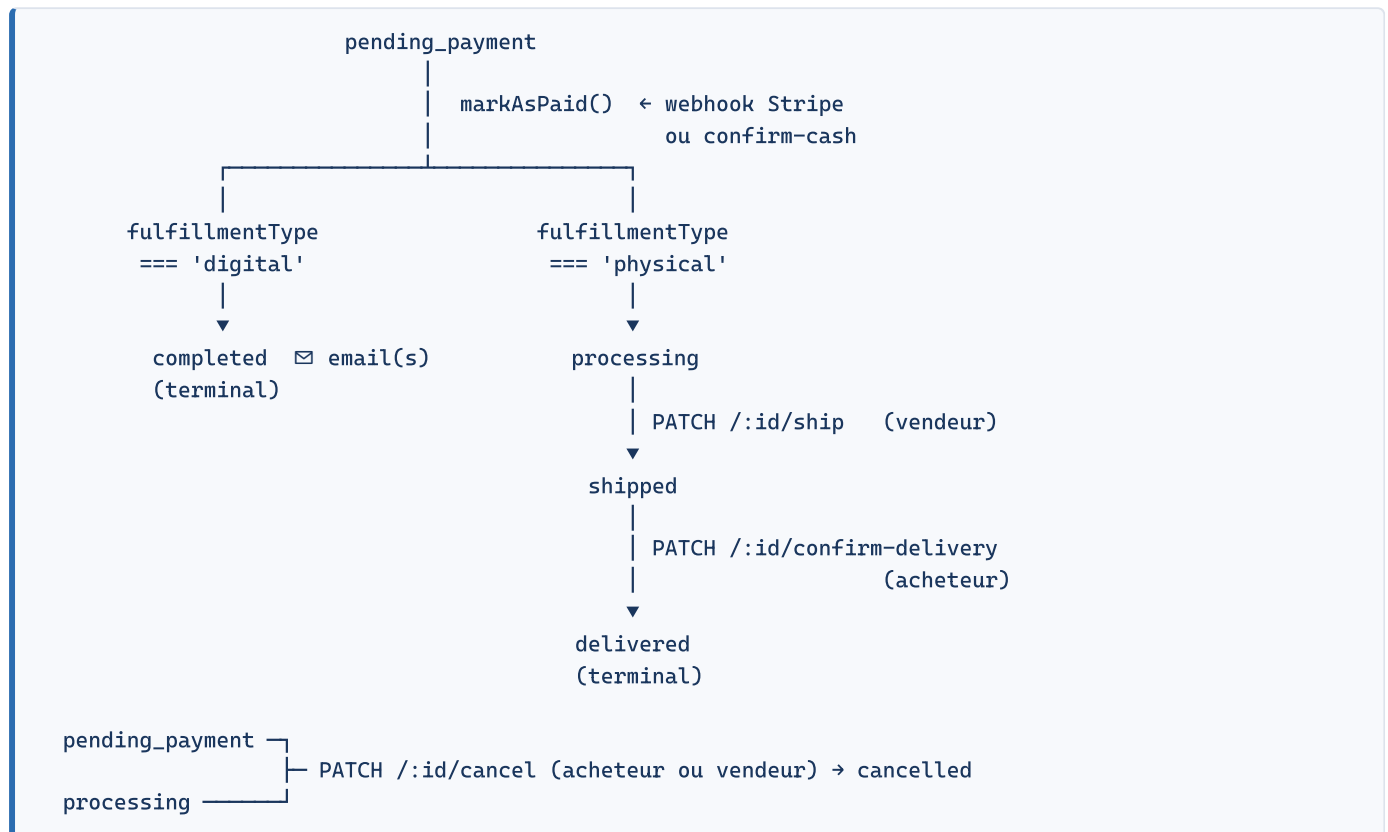
- |— Commande 1 · vendeur A · physical · [Clavier, Souris]
- |— Commande 2 · vendeur A · digital · [Ebook]
- |— Commande 3 · vendeur B · physical · [Tapis]

Trois commandes, donc trois paiements séparés

Un seul « passer commande » côté utilisateur produit N commandes, chacune avec son propre `total` et son propre `PaymentIntent` Stripe. Le frontend doit gérer ce cas : soit enchaîner les paiements, soit présenter une page récapitulative listant les commandes créées avec un bouton « Payer » par ligne. C'est une contrainte structurelle, pas un détail d'implémentation — anticipez-la dès la maquette.

C'est également pendant cette boucle que le **stock est réservé**, article par article, pour les seuls produits physiques. Si un seul article manque, tout le checkout est refusé en **400** et les réservations déjà prises sont rendues — on n'obtient jamais une commande partielle. Voir §7 n° 5 pour le détail du mécanisme de compensation.

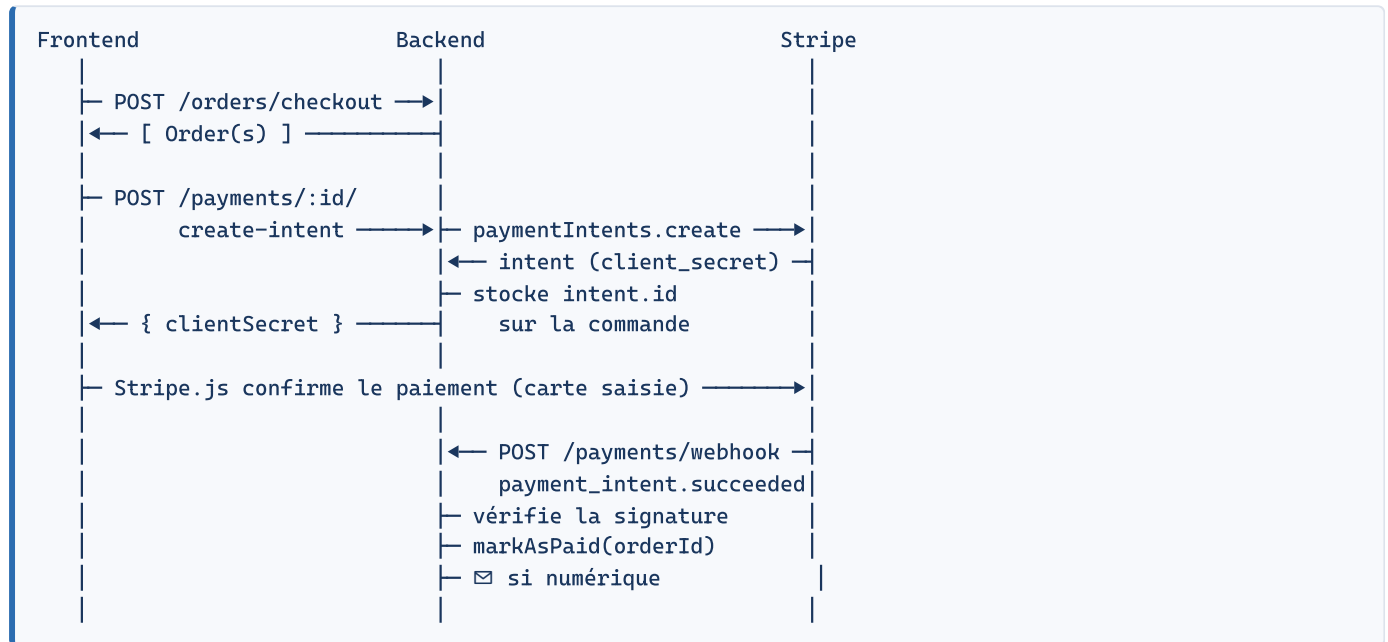
6.2 Machine à états d'une commande



Statut	Concerne	Signification
<code>pending_payment</code>	les deux	Créée au checkout, en attente de règlement
<code>processing</code>	physique	Payée, le vendeur doit préparer et expédier
<code>shipped</code>	physique	Expédiée par le vendeur
<code>delivered</code>	physique	Réception confirmée par l'acheteur — terminal
<code>completed</code>	numérique	Payée et livrée par email — terminal
<code>cancelled</code>	les deux	Annulée avant expédition

Notez l'asymétrie : le parcours physique se termine sur `delivered`, le parcours numérique sur `completed`. Deux statuts terminaux différents pour un même concept de « commande aboutie ». Votre affichage frontend doit traiter les deux comme un succès — ne filtrez pas sur `completed` seul, vous manqueriez toutes les commandes physiques.

6.3 Paiement par carte (Stripe)



Trois points à retenir pour l'implémentation frontend :

- L' `orderId` voyage dans `metadata` du `PaymentIntent`. C'est ainsi que le webhook retrouve la commande — le frontend n'a rien à faire pour cela.
- Les montants sont convertis en centimes (`Math.round(total * 100)`), comme l'exige Stripe.
- **Le webhook est asynchrone.** Quand Stripe.js confirme côté client, la commande n'est pas encore passée en `processing` — le webhook peut arriver une ou deux secondes plus tard. Ne redirigez pas vers une page qui affiche le statut brut en supposant qu'il est déjà à jour ; prévoyez un état « paiement en cours de confirmation », avec relance périodique ou message d'attente.

L'idempotence est correctement gérée

`markAsPaid()` commence par `if (order.status !== 'pending_payment') return order;`. Stripe pouvant livrer un même webhook plusieurs fois, cette garde évite qu'une commande soit traitée deux fois et que les emails de livraison numérique partent en double.

6.4 Paiement en espèces

Beaucoup plus simple, et pensé pour la remise en main propre. La commande est créée en `pending_payment`, puis le **vendeur** appelle `PATCH /payments/:orderId/confirm-cash` après avoir reçu l'argent. Le même `markAsPaid()` est invoqué — donc rigoureusement le même comportement en aval, y compris l'envoi des emails pour les produits numériques.

Une faiblesse de confiance à connaître

Rien ne vérifie que l'acheteur a réellement payé. Le vendeur seul atteste. Pour un MVP entre utilisateurs qui se rencontrent, cela se défend ; à terme, une double confirmation acheteur + vendeur serait plus robuste.

6.5 Livraison d'un produit numérique

Déclenchée automatiquement dans `markAsPaid()` lorsque `fulfillmentType === 'digital'`. La méthode privée `deliverDigitalItems()` récupère l'acheteur, parcourt les lignes de commande et envoie **un email par article**

possédant un `fileKey`.

Le `fileKey` est envoyé tel quel comme lien

`sendDigitalDelivery(buyer.email, item.title, product.fileKey)` — le troisième paramètre s'appelle `downloadUrl` et atterrit directement dans un `<a href>`. Or le schéma décrit explicitement `fileKey` comme « référence de stockage, jamais une URL publique ». Il manque la couche qui transforme cette référence en URL signée à durée limitée. De plus, `downloadLimit` est stocké mais aucun compteur ne le fait respecter : le lien est réutilisable indéfiniment et partageable.

7. Corrections appliquées

Les huit points relevés lors de l'audit initial ont tous été corrigés. Cette section décrit ce qui a changé et pourquoi — elle vaut autant comme journal de décisions que comme vérification. L'ensemble a été validé par **39 tests de bout en bout** contre la pile réelle (Mongo, Redis, API).

✓ N° 1 — CORS activé

```
// src/main.ts
app.enableCors({
  origin: process.env.FRONTEND_URL || 'http://localhost:5173',
  credentials: true,
});
```

`FRONTEND_URL` a été ajouté à `.env.example` et au `docker-compose.yml`. Le choix de `||` plutôt que `??` est volontaire : docker-compose transmet une chaîne vide lorsqu'une variable est absente du `.env`, et `??` ne rattrape que `null` et `undefined` — l'origine autorisée aurait alors été `""`, bloquant tout.

✓ N° 2 — Publication des produits

Une route dédiée a été retenue plutôt que l'ouverture du champ `status` dans le DTO : publier est une action métier distincte d'une édition de contenu, et elle mérite son propre bouton dans l'interface.

```
PATCH /products/:id/status { status: 'draft' | 'active' }
```

`'suspended'` est délibérément absent de `UpdateProductStatusDto` : c'est un statut de **modération**, il n'appartient pas au vendeur de se suspendre ou de se réactiver lui-même. Une tentative renvoie un **400**. La route filtre sur `{ _id, sellerId }`, donc un vendeur ne peut publier que ses propres produits.

✓ N° 3 — Tableau de bord vendeur

```
GET /products/mine → tous statuts, brouillons inclus
```

Déclarée **avant** `@Get('/:id')` dans le contrôleur — le journal de démarrage confirme l'ordre de résolution :

```
Mapped {/products, GET} route
Mapped {/products/mine, GET} route ← avant
Mapped {/products/:id, GET} route ← après
```

Contrairement au catalogue public, cette route renvoie un tableau simple et non paginé : un vendeur gère un nombre de produits qui tient sur un écran, et la pagination compliquerait inutilement l'interface de gestion.

✓ N° 4 — Upload verrouillé

```
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('seller')
@UseInterceptors(FileInterceptor('image', {
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (_req, file, cb) => {
    if (!/^image\/(jpeg|png|webp)$/.test(file.mimetype)) {
      return cb(new BadRequestException('Format non supporté...'), false);
    }
    cb(null, true);
  },
}))
```

Passer une *erreur* au callback plutôt que `cb(null, false)` produit un **400** explicite. Avec `false`, Multer écarte silencieusement le fichier et le contrôleur répondrait « aucun fichier reçu » — un message trompeur qui enverrait l'utilisateur chercher au mauvais endroit.

✓ N° 5 — Gestion du stock

Le décrément est **atomique** : la condition fait partie du filtre de la requête.

```
async decrementStock(productId: string, quantity: number): Promise<boolean> {
  const result = await this.productModel.updateOne(
    { _id: productId, stock: { $gte: quantity } }, // ← la garde est dans le filtre
    { $inc: { stock: -quantity } },
  ).exec();
  return result.modifiedCount === 1;
}
```

Deux commandes simultanées ne peuvent donc pas faire passer le stock sous zéro : MongoDB applique les deux `updateOne` en série, et la seconde ne trouve aucun document correspondant. Une vérification en deux temps (lire puis écrire) laisserait au contraire une fenêtre de concurrence.

`checkout()` réserve le stock article par article, avec compensation en cas d'échec :

```
const reserved = [];
let ordersCreated = false;
try {
  for (const cartItem of cart.items) {
    if (product.type === 'physical') {
      const ok = await this.productsService.decrementStock(/* ... */);
      if (!ok) throw new BadRequestException('Stock insuffisant pour « ${product.title} »');
      reserved.push({ productId, quantity });
    }
  }
  /* ... regroupement par vendeur + type ... */
}
const orders = await Promise.all(/* ... */);
ordersCreated = true;
await this.cartService.clearCart(buyerId);
return orders;
} catch (err) {
  if (!ordersCreated) await this.releaseReservations(reserved);
  throw err;
}
```

Pourquoi le drapeau `ordersCreated` ?

Une fois les commandes enregistrées, la réservation de stock leur appartient. Restituer le stock à ce stade créerait exactement la survente que l'on cherche à empêcher : les articles seraient remis en rayon alors qu'ils sont déjà vendus. La compensation ne doit donc s'appliquer qu'aux échecs survenus *avant* la création des commandes.

L'annulation restitue symétriquement le stock. Les deux seuls statuts annulables — `pending_payment` et `processing` — sont tous deux postérieurs au checkout, donc le stock a bien été décompté dans les deux cas.

✓ N° 6 — Le prix promotionnel est appliqué

```
const hasPromo = product.promoPrice !== null && product.promoPrice < product.price;
const unitPrice = hasPromo ? product.promoPrice! : product.price;

items.push({
  price: unitPrice, // ce qui est facturé
  ...(hasPromo ? { originalPrice: product.price } : {}), // le prix barré
  subtotal: unitPrice * cart[productId],
});
```

La convention retenue : `price` est **toujours** le montant réellement facturé, et `originalPrice` n'apparaît que lorsqu'une promotion s'applique. Le frontend n'a donc aucun arbitrage à faire — il affiche `price`, et barre `originalPrice` s'il est présent.

Le test `promoPrice < price` évite qu'une « promotion » saisie par erreur au-dessus du prix normal ne majore la facture. Comme `OrderItem` fige `cartItem.price` au checkout, c'est bien le prix promo qui est gravé dans la commande.

✓ N° 7 — Les emails ne font plus échouer les opérations

À l'inscription, l'échec d'envoi est journalisé sans interrompre le flux : le compte étant déjà créé, propager l'erreur afficherait un échec à un utilisateur dont le compte existe — et qui ne pourrait plus se réinscrire (409 Conflict).

```
try {
  await this.emailService.sendWelcomeEmail(user.email, user.name);
} catch (err) {
  this.logger.warn(`Email de bienvenue non envoyé à ${user.email}`, /* ... */);
}
```

Dans `deliverDigitalItems()`, le `try/catch` est placé **à l'intérieur** de la boucle : un envoi défaillant ne prive plus l'acheteur des articles suivants de la même commande.

✓ N° 8 — Points secondaires

Point	Correction
<code>shippingOptions</code>	Ajouté au DTO via <code>ShippingOptionDto</code> + <code>@ValidateNested</code> et <code>@Type</code> . Facultatif même en physique : un vendeur peut ne proposer qu'un retrait en main propre
Pagination	<code>?page=</code> / <code>?limit=</code> (défauts 1 / 20, plafond 100). Réponse <code>{ items, total, page, limit, totalPages }</code>
Recherche	<code>?q=</code> sur titre et description. Regex insensible à la casse plutôt qu'un index <code>\$text</code> : elle trouve les correspondances partielles (« clav » → « clavier »), là où un index texte n'indexe que des mots entiers. Les métacaractères sont échappés
Relations peuplées	<code>categoryId</code> (name, slug) et <code>sellerId</code> (name) sur les routes d'affichage. Volontairement pas sur <code>findOne()</code> interne — voir l'encadré ci-dessous
Suppression de catégorie	400 si elle contient des sous-catégories ou des produits, avec le décompte dans le message
<code>DELETE /cart</code>	Renvoie désormais <code>{ items: [], total: 0 }</code> , comme les quatre autres routes du panier

Piège évité : `populate` casse `.toString()`

`ProductsService.findOne()` est utilisé en interne par `CartService` et `OrdersService`, qui lisent `product.sellerId.toString()` pour regrouper les commandes. Sur un champ peuplé, `toString()` ne renvoie plus l'`ObjectId` mais la représentation du document — le regroupement par vendeur aurait silencieusement volé en éclats. D'où deux méthodes distinctes : `findOne()` reste brute pour les appels internes, `findOneDetailed()` peuple pour l'affichage.

Couverture des tests

39 assertions exécutées contre la pile réelle, toutes vertes : révocation de token au logout, refus de `/categories` à un vendeur (403), invisibilité puis publication d'un brouillon, refus de `'suspended'`, pagination et `page=0` rejeté, recherche partielle, peuplement des relations, upload refusé sans token (401) / à un acheteur (403) / en PDF (400), prix promo facturé à 80 au lieu de 100, décrétement 3→1 au checkout, restitution à 3 après annulation, survente refusée avec compensation vérifiée, et suppression de catégorie bloquée dans les deux cas de figure.

8. Structure de frontend recommandée

Ce qui est déjà installé

`frontend/src/` ne contient aujourd'hui que le squelette Vite (`App.tsx` , `main.tsx` , deux CSS, des assets). Les dépendances, elles, sont déjà choisies — et bien choisies :

Paquet	Rôle attendu
React 19 + Vite 8	Socle et outillage de build
<code>@reduxjs/toolkit</code> + <code>react-redux</code>	État global — et RTK Query pour l'appel API
<code>react-router-dom</code> 7	Routage et protection des routes
<code>react-hook-form</code> + <code>zod</code> + <code>@hookform/resolvers</code>	Formulaires et validation client
Tailwind CSS 4 (<code>@tailwindcss/vite</code>)	Styles

Recommandation : utilisez RTK Query, pas des `fetch` manuels

Redux Toolkit est déjà installé, et RTK Query en fait partie sans dépendance supplémentaire. Il vous donne le cache, la déduplication des requêtes, l'invalidation par tags et les états de chargement — tout ce que vous réécririez à la main autrement. Un point particulièrement pertinent ici : les routes du panier renvoyant l'état complet, vous pouvez mettre le cache à jour directement dans `onQueryStarted` au lieu d'invalider et recharger.

Arborescence proposée

```
frontend/src/
├── app/
│   ├── store.ts           configureStore + middleware RTK Query
│   └── hooks.ts           useAppDispatch / useAppSelector typés
├── features/              une tranche verticale par domaine backend
│   ├── auth/              authApi.ts, authSlice.ts, LoginForm, RegisterForm
│   ├── products/          productsApi.ts, ProductCard, ProductGrid, ProductForm
│   ├── categories/        categoriesApi.ts, CategoryTree, CategoryFilter
│   ├── cart/              cartApi.ts, CartDrawer, CartItemRow
│   ├── orders/            ordersApi.ts, OrderCard, OrderStatusBadge, OrderTimeline
│   └── payments/          paymentsApi.ts, StripeCheckout, CashPaymentNotice
├── components/            UI réutilisable, sans logique métier
│   ├── ui/                Button, Input, Modal, Badge, Spinner, EmptyState
│   └── layout/            Header, Footer, Sidebar, RoleNav
├── routes/
│   ├── router.tsx         createBrowserRouter
│   ├── ProtectedRoute.tsx exige un token
│   └── RoleRoute.tsx      exige un rôle précis
├── pages/
│   ├── public/            Home, ProductList, ProductDetail, Login, Register
│   ├── buyer/            Cart, Checkout, MyPurchases, OrderDetail
│   ├── seller/            Dashboard, MyProducts, ProductEditor, MySales
│   └── admin/             Categories
├── lib/
│   ├── apiClient.ts       baseQuery : URL de base + en-tête Bearer
│   ├── schemas.ts         schémas Zod alignés sur les DTO backend
│   └── orderStatus.ts     libellés FR + couleurs par statut
└── types/api.ts           types TS miroirs des schémas Mongoose
```

Points de câblage à ne pas manquer

1. L'injection du token

Centralisez-la dans le `baseQuery` de RTK Query, jamais appel par appel :

```
const baseQuery = fetchBaseQuery({
  baseUrl: import.meta.env.VITE_API_URL, // http://localhost:3000
  prepareHeaders: (headers, { getState }) => {
    const token = (getState() as RootState).auth.token;
    if (token) headers.set('Authorization', `Bearer ${token}`);
    return headers;
  },
});
```

2. La déconnexion globale sur 401

Sans refresh token, un 401 signifie toujours « session finie ». Enveloppez le `baseQuery` pour purger l'état et rediriger :

```
const baseQueryWithAuth: BaseQueryFn = async (args, api, extra) => {
  const result = await baseQuery(args, api, extra);
  if (result.error?.status === 401) {
    api.dispatch(login()); // vide le store + localStorage
  }
  return result;
};
```

3. Trois espaces distincts, pilotés par le rôle

`GET /auth/me` renvoie le rôle. Faites-en la clé de votre navigation : un acheteur ne doit pas voir « Mes ventes », un vendeur doit avoir un tableau de bord produits, l'admin la gestion des catégories. `RoleRoute` protège les routes, mais masquez aussi les liens — une route protégée dont le lien est visible est une mauvaise expérience.

4. Le formulaire produit est le morceau le plus délicat

Il doit refléter la validation conditionnelle du backend. Avec Zod, un `discriminatedUnion` exprime exactement la même règle :

```
const productSchema = z.discriminatedUnion('type', [
  z.object({
    type: z.literal('physical'),
    stock: z.number().int().min(0),
    weightGrams: z.number().int().min(0),
    /* ...champs communs */
  }),
  z.object({
    type: z.literal('digital'),
    fileKey: z.string().min(1),
    downloadLimit: z.number().int().min(1),
    /* ...champs communs */
  }),
]);
```

Et n'envoyez que les champs du variant actif : `forbidNonWhitelisted` rejettera un `stock` traînant sur un produit numérique.

5. L'upload d'image précède la création du produit

Le flux est en deux temps : `POST /upload/image` renvoie une URL, que vous placez ensuite dans le tableau `images` du `POST /products`. Prévoyez un composant d'upload autonome, avec prévisualisation et suppression, dont la valeur de sortie est un `string[]` d'URLs.

6. Le checkout renvoie un tableau

Rappel de la section 6.1 : ne codez pas `const order = response[0]`. Après `POST /orders/checkout`, redirigez vers un récapitulatif listant **toutes** les commandes créées, avec pour chacune son montant et son action de paiement.

Ordre de construction suggéré

1. **Socle** : store Redux, `baseQuery` avec token, router, `ProtectedRoute`, layout et navigation.
2. **Authentification** : inscription, connexion, `/auth/me` au démarrage, déconnexion. C'est la porte d'entrée de tout le reste.
3. **Catalogue public** : liste des produits, filtre par catégorie, page produit. Aucun token nécessaire — vous validez le socle sans dépendre de l'auth.
4. **Panier** : ajout, modification, suppression, badge de quantité.
5. **Espace vendeur** : mes produits, création avec upload, publication.
6. **Commandes** : checkout, mes achats, mes ventes, transitions de statut.
7. **Paiement Stripe** : en dernier — c'est le plus dépendant de tout le reste, et le plus pénible à tester.

En résumé

Le backend est solidement structuré : séparation nette des modules, validation stricte, règles métier centralisées au bon endroit, instantanés de commande bien pensés, idempotence du webhook gérée. Les huit points relevés à l'audit ont tous été corrigés et vérifiés (section 7) — aucun ne remettait en cause l'architecture. Le contrat d'API est stable : vous pouvez attaquer le frontend dans l'ordre proposé ci-dessus.

Ce qui reste hors périmètre

Deux points connus n'ont volontairement pas été traités, car ils dépassent la correction de bugs. **Les liens de téléchargement numériques** (§6.5) exposent toujours le `fileKey` brut : il faudrait des URLs signées à durée limitée et un compteur appliquant `downloadLimit`. **Les frais de port** sont désormais saisissables via `shippingOptions`, mais ne sont ni choisis au checkout ni ajoutés au `total` — le modèle de commande n'a pas de champ pour cela. Les deux méritent d'être décidés avant la mise en production.